



HOJA DE RETO

Módulo 3 · Generación y Optimización de Código

StockGuard — QA, Documentación y Pipeline CI/CD con Python e IA

DATOS DE IDENTIFICACIÓN

Nombre de empresa colaboradora:	Fundación Dicampus
Nombre del alumno/a:	Angel Echenique
Módulo / Programa:	Módulo 3 · Generación y Optimización de Código
Modalidad:	Individual
Fecha de presentación:	07/04/2026
Fecha límite de entrega:	10 / 04 / 2026
Enlace GitHub del repositorio:	https://github.com/kindred-98/StockGuard

DESCRIPCIÓN DEL RETO / PROBLEMA

Contexto real: El software actual de gestión de inventario de Fundación Dicampus presenta fallos aleatorios en el cálculo de existencias y carece por completo de validaciones de seguridad. Esto provoca pérdidas económicas, inconsistencias en los datos y desconfianza de los usuarios. El código heredado no es mantenible porque no tiene documentación, ni pruebas automáticas, ni ningún mecanismo que impida introducir valores absurdos — como stock negativo o precios de cero euros.



Queremos que diseñes y ejecutes un **proyecto piloto individual de QA y seguridad de código** aplicando Inteligencia Artificial como copiloto: el sistema se llama **StockGuard**, un gestor de existencias en consola escrito en Python 3.12+.

Partirás de un fichero **stockguard.py** heredado, con vulnerabilidades conocidas, y lo transformarás aplicando los cuatro pilares del módulo: **auditoría de seguridad**, **documentación técnica**, **testing con mocks** y **pipeline CI/CD automatizado**. El objetivo es demostrar que la IA puede apoyar todo el ciclo de calidad de software — desde la detección de errores hasta la protección continua del repositorio — sin sustituir el criterio del desarrollador/a.

El resultado final deberá subirse a un repositorio público de GitHub, estar completamente documentado y tener el pipeline de integración continua en verde antes de la entrega.

¿PARA QUIÉN ES UN PROBLEMA?

¿Quién es el público objetivo principal?

Departamento de TI / Desarrollo de Software: Desarrolladores que heredan código legacy y deben mantenerlo, mejorarlo

y asegurarlo. Necesitan herramientas y procesos que garanticen la calidad del código que entregan.

¿Qué otros perfiles están implicados? (usuarios, responsables técnicos, gestores, etc.)

PERFIL	NECESIDAD
Usuarios del sistema de inventario	Dependientes de que el sistema funcione correctamente para gestionar existencias y evitar pérdidas.
Responsables técnicos / Team Leads	Necesitan visibilidad de la calidad del código y métricas objetivas.
Gestores de proyecto	Requieren assurance de que el software cumple estándares de calidad antes de producción.
Audidores de seguridad	Necesitan evidencia de que el código ha sido revisado y hardening aplicado.

DevOps / CI Engineers	Requieren pipelines automatizados que garanticen calidad en cada despliegue.
-----------------------	--

¿POR QUÉ ES IMPORTANTE RESOLVER EL RETO?

¿Qué consecuencias tiene actualmente no tener validaciones ni tests en el sistema de inventario?

Consecuencias de no tener validaciones ni tests:

- **Inconsistencia de datos:** Permite valores físicamente imposibles como stock negativo (-5 unidades) o precios de cero, corrompiendo la integridad de los datos.
- **Pérdidas económicas:** Cálculos incorrectos de valoración de inventario (qty × price) resultan en informes financieros erróneos, afectando la toma de decisiones.
- **Fallos en producción:** Sin tests, cualquier cambio puede introducir regresiones que solo se detectan en producción, generando downtime y coste de corrección.
- **Brechas de seguridad:** Sin validación de entradas, un atacante podría manipular precios negativos para generar abonos automáticos o manipular balances.
- **Imposibilidad de mantenimiento:** Código sin documentación ni tests es dependiente del autor original. Cualquier refactorización es de alto riesgo.
- **Incumplimiento normativo:** En entornos regulados (sanidad, alimentación), la trazabilidad y validación de datos es un requisito legal.

¿Qué beneficios aportaría resolver este reto a los distintos actores implicados?

ACTOR	BENEFICIOS
Desarrolladores	Confianza al hacer cambios, documentación clara, tests que sirven como especificación ejecutable.



Hoja de reto



Usuarios	Sistema más fiable, menos errores, datos consistentes.
Team Leads	Métricas objetivas de calidad, visibilidad del estado del código
Gestores	Reducción de costes de mantenimiento, menor tiempo de corrección de bugs.
Audidores	Evidencia de código seguro y testeado, compliance demostrado
DevOps	Pipeline automatizado que filtra código de baja calidad antes de producción.

¿QUÉ RESULTADOS ESPERAMOS OBTENER?

Al finalizar el reto, la empresa colaboradora espera obtener los siguientes entregables:

Entregable	Descripción	
Código auditado	Archivo AUDIT.md con las 3 vulnerabilidades identificadas, su riesgo y la corrección aplicada.	■ Completado
Modelo validado	Clase Item con dataclass, type hints y validaciones que impiden valores negativos o nulos.	■ Completado
Documentación técnica	Docstrings en formato Google en todas las funciones públicas + README completo con badge CI.	■ Completado
Batería de tests	Tests con pytest y pytest-mock que cubren casos válidos, edge cases y simulación de archivo corrupto.	■ Completado (34 TESTS)
Pipeline CI/CD	Archivo ci.yml en GitHub Actions que ejecuta linter (flake8) y tests en cada push a main.	■ Completado
Pipeline Security	Archivo security.yml con Bandit para escaneo de vulnerabilidades de seguridad.	■ Completado



Hoja de reto



Análisis SonarCloud	Integración con SonarCloud para métricas de calidad continuas.	■ Completado
Repositorio público	Repo en GitHub con mínimo 12 commits etiquetados, pipeline en verde y README con capturas.	■ Completado

¿Qué resultados adicionales esperas como empresa a medio/largo plazo?

- **Adopción de estándares de calidad:** El repositorio sirve como template para futuros proyectos, estableciendo el estándar de documentación, testing y CI/CD en la organización.
- **Cultura de seguridad integrada:** La inclusión de Bandit y SonarCloud en el pipeline demuestra que la seguridad no es un afterthought, sino parte del ciclo de desarrollo.
- **Automatización completa:** Con los pipelines funcionando, cualquier push o PR activa verificaciones automáticas, reduciendo la carga manual de QA.
- **Mantenibilidad demostrada:** El código documentado y testeado podrá ser heredado por cualquier desarrollador sin dependencia del autor original.
- **Escalabilidad del proceso:** Las prácticas aprendidas (auditoría con IA, generación de tests, documentación automatizada) son transferibles a otros sistemas de la fundación.

ENTREGABLES ADICIONALES IMPLEMENTADOS

Más allá de los requisitos mínimos, este proyecto incluye:



Hoja de reto



ENTREGABLE	DESCRIPCION
Pre-commit hooks	Configuración de flake8 y black para ejecución local antes de commit.
Coverage reporting	pytest-cov con objetivo $\geq 80\%$ (llegamos a 97%).
Documentación de pipelines	Archivo dedicado explicando interacción entre CI, Security y SonarCloud.
Informe completo de desarrollo	Documentación detallada de cada fase y decisión de diseño.
Reflexión de uso de IA	Tabla documentando prompts utilizados y modificaciones realizadas.

RESUMEN DE MÉTRICAS FINALES

METRICA	VALOR	OBJETIVO
Tests unitarios	34	≥ 20
Coverage	97%	$\geq 80\%$
Code Smells	0	0
Vulnerabilidades	0	0
Bugs	0	0
Quality Gate	■ Passed	Passed
Pipelines en verde	3/3	2/2 MINIMO

PLAZO DE PRESENTACIÓN



Hoja de reto



Hito	Fecha límite
Commit 12 — entrega final (README + reflexión)	10/ 04 / 2026

Uso responsable de IA: Cada commit donde participó la IA debe llevar la etiqueta [ai]. Debes poder explicar en voz alta todo el código de tu repositorio. El plagio de código sin comprensión implica suspensión del reto.